

Homework 4

Benjamin Pedersen

Part A: Exercises for Thall and Vail data

```
# Fitting Poisson model
summary_f1 <- summary(f1 <- glm(seiz ~ log(age) +
                               log_base +
                               treat,
                               data = seiz_total,
                               family = poisson()))

summary_f1

##
## Call:
## glm(formula = seiz ~ log(age) + log_base + treat, family = poisson(),
##      data = seiz_total)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -6.0834  -2.0602  -0.4096   1.3963   8.1997
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.02151    0.40354  -2.531  0.0114 *
## log(age)     0.58778    0.10992   5.347 8.93e-08 ***
## log_base     1.22522    0.03252  37.672 < 2e-16 ***
## treat       -0.01759    0.04818  -0.365  0.7150
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 2122.73  on 58  degrees of freedom
## Residual deviance:  556.39  on 55  degrees of freedom
## AIC: 847.66
##
## Number of Fisher Scoring iterations: 5

# Fitting Quasi-Poisson model, estimates should be the same, but the variance will be different
summary_fq1 <- summary(fq1 <- glm(seiz ~ log(age) +
                                  log_base +
                                  treat,
                                  data = seiz_total,
                                  family = quasipoisson()))

summary_fq1
```

```
##
## Call:
## glm(formula = seiz ~ log(age) + log_base + treat, family = quasipoisson(),
##      data = seiz_total)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -6.0834  -2.0602  -0.4096   1.3963   8.1997
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.02151    1.34172  -0.761   0.450
## log(age)      0.58778    0.36548   1.608   0.114
## log_base      1.22522    0.10813  11.330 5.27e-16 ***
## treat         -0.01759    0.16019  -0.110   0.913
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for quasipoisson family taken to be 11.05488)
##
##      Null deviance: 2122.73  on 58  degrees of freedom
## Residual deviance:  556.39  on 55  degrees of freedom
## AIC: NA
##
## Number of Fisher Scoring iterations: 5
```

1. Verify that the variances of the coefficients under the Quasi-Poisson model are inflated from the variances of the standard Poisson regression model by a constant factor.

As seen before, we get the same SEs as in the Quasi-Poisson model. # to here

```
se_poisson <- sqrt(diag(vcov(f1))) # Calculate Poisson SE
se_qpoisson <- sqrt(diag(vcov(fq1))) # Calculate Quasi-Poisson SE
var_diff <- se_poisson / se_qpoisson # dividing vectors, should get the
var_diff                                # same values for each term
```

```
## (Intercept)    log(age)    log_base      treat
##      0.300762    0.300762    0.300762    0.300762
```

As seen above, we get the same standard errors as the from the Quasi-Poisson model and the variances are inflated by a constant factor. If we divide the Poisson model's standard error by the Quasi-Poisson model's standard error, it produces the same constant for each term (intercept, log(age), log_base, and treatment).

2. Use a non-parametric bootstrap to obtain standard error estimates for all coefficients in the above-fitted Poisson GLM. How do the results compare with the standard Poisson GLM? How do the results compare with the quasi-Poisson model?

```
set.seed(1) # for reproducible results
beta_boot <- replicate(2000, {
  boot_sample <- sample_n(seiz_total, # Sample data with replacement
                          size = nrow(seiz_total),
                          replace = TRUE)
  fq1_boot <- glm(seiz ~ log(age) + # Fitting Quasi-Poisson model
                 log_base +
```

```

        treat,
        data = boot_sample,
        family = poisson())
beta <- coefficients(fq1_boot) # Extract and return coefficients
beta
})

boot_se <- cbind(intercept_sd = sd(beta_boot[1,]), # calculating SE for each
                 log_age_sd = sd(beta_boot[2,]),   # variable
                 log_base_sd = sd(beta_boot[3,]),
                 treat_sd = sd(beta_boot[4,]))
boot_se

##      intercept_sd log_age_sd log_base_sd treat_sd
## [1,]      1.115793  0.3096452  0.1714965 0.2086041

```

While the bootstrap standard errors are much larger than the Poisson model's, at least twice the value, the bootstrap's standard errors are fairly similar to the Quasi-Poisson model's with a variation between variables being lower and higher.

3. Use the bootstrap to estimate the bias for each coefficient estimate – are the biases small or large?

Using the formula:

$$\widehat{Bias}^* = \overline{\hat{\theta}^*} - \hat{\theta}$$

```

intercept_bias <- mean(beta_boot[1,] - summary_fq1$coefficients[1, 1])
log_age_bias <- mean(beta_boot[2,] - summary_fq1$coefficients[2, 1])
log_base_bias <- mean(beta_boot[3,] - summary_fq1$coefficients[3, 1])
treat_bias <- mean(beta_boot[4,] - summary_fq1$coefficients[4, 1])

```

Intercept bias: 0.0359872

Log(age) bias: 0.0225051

Log_base bias: -0.0478193

Treatment bias: -0.0620333

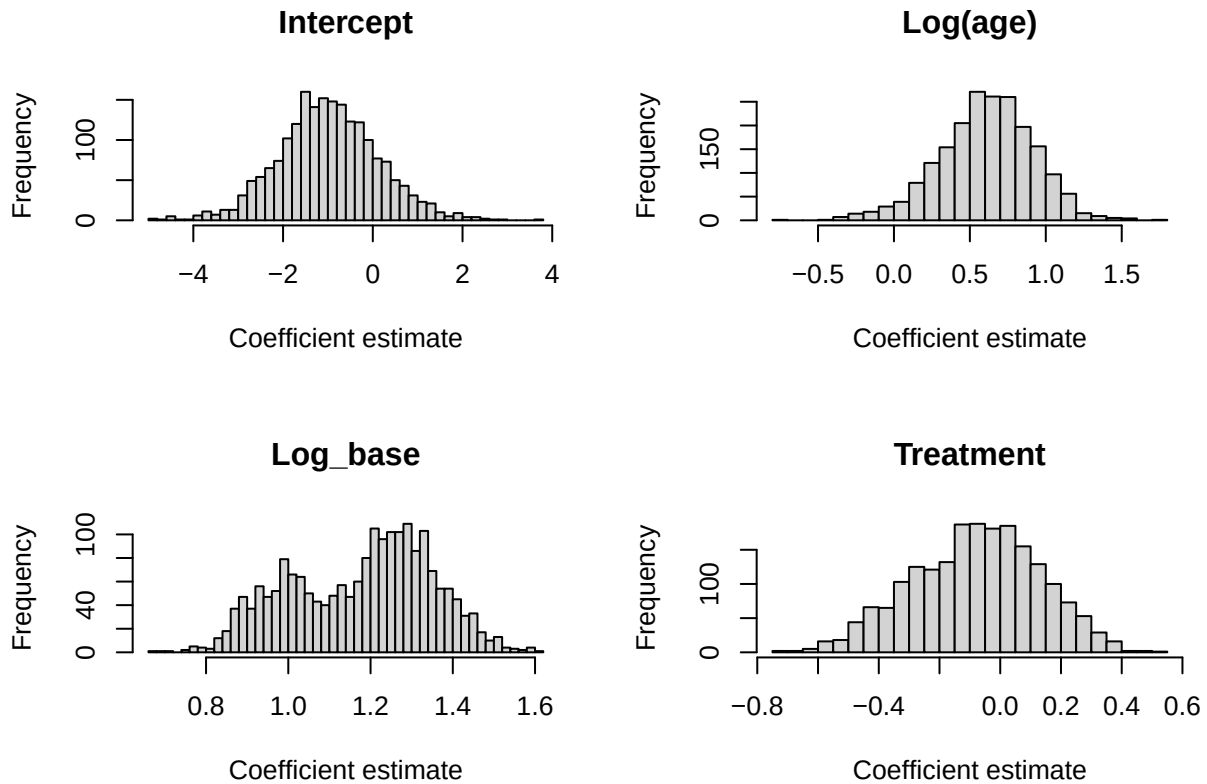
The bias for each covariate appears to be pretty small.

4. Visualize the bootstrap sampling distribution of each of the coefficients – does a normal approximation seem reasonable for all coefficients? Construct bootstrap 95% CI's for each coefficient using the i) normal approximation method and ii) the percentile method and compare the results – if they are substantially different, why do you think that is the case? Hint: there may be more than just 1 reason.

```

par(mfrow=c(2,2))
hist(beta_boot[1,], breaks = 35, xlab = "Coefficient estimate", main = "Intercept")
hist(beta_boot[2,], breaks = 35, xlab = "Coefficient estimate", main = "Log(age)")
hist(beta_boot[3,], breaks = 35, xlab = "Coefficient estimate", main = "Log_base")
hist(beta_boot[4,], breaks = 35, xlab = "Coefficient estimate", main = "Treatment")

```



```
ci_norm_approx <- rbind( # CI Normal Approximation Method:
  LCI = coefficients(fq1) - qt(0.975, nrow(seiz_total) - 1) * boot_se,
  UCI = coefficients(fq1) + qt(0.975, nrow(seiz_total)-1) * boot_se
)
round(ci_norm_approx, 2)
```

```
##      intercept_sd log_age_sd log_base_sd treat_sd
## [1,]      -3.26      -0.03         0.88      -0.44
## [2,]       1.21       1.21         1.57       0.40
```

```
intercept = quantile(beta_boot[1,], probs = c(0.025, 0.975)) # calculate CIs
log_age = quantile(beta_boot[2,], probs = c(0.025, 0.975)) # for percentile
log_base = quantile(beta_boot[3,], probs = c(0.025, 0.975)) # bootstrap method
treatment = quantile(beta_boot[4,], probs = c(0.025, 0.975))

boot_percentile_ci <- cbind(intercept, # bind variable CIs
                             log_age,
                             log_base,
                             treatment)
round(boot_percentile_ci, 2) # print and round values
```

```
##      intercept log_age log_base treatment
## 2.5%      -3.16   -0.06      0.86      -0.49
## 97.5%       1.27    1.17      1.46       0.30
```

While the Normal Approximation Method seems reasonable for intercept and log(age), it does not seem reasonable for the log-base estimate as it is bimodal. This is problematic because if the bootstrap distribution of the estimated coefficient is not approximately normal, the confidence intervals via the Normal Approximation Method may not produce accurate results. Because of this, the Percentile Method may be more appropriate.

5. Repeat questions 2. and 4. above but using the following model that includes all possible interactions of the three predictors: `glm(seiz ~ log(age)log_basetreat, data = seiz_total, family = poisson())`. Describe any differences in the bootstrap sampling distributions of the coefficients compared with the first model fit.

```
set.seed(1)

fq2 <- glm(seiz ~ log(age) * log_base * treat, # Poisson interaction model
           data = seiz_total,
           family = poisson())

beta_boot2 <- replicate(2000, {
  boot_sample <- sample_n(seiz_total, # sample data w/ replacement
                          size = nrow(seiz_total),
                          replace = TRUE)
  fq2_boot <- glm(seiz ~ log(age) * log_base * treat, # quasi-poisson
                  data = boot_sample,                # interaction model
                  family = poisson())
  beta <- coefficients(fq2_boot) # extract and return coefficients
  beta
})
```

```
intercept_sd <- sd(beta_boot2[1,])
log_age_sd <- sd(beta_boot2[2,])
log_base_sd <- sd(beta_boot2[3,])
treat_sd <- sd(beta_boot2[4,])
log_age_log_base_sd <- sd(beta_boot2[5,])
log_age_treat_sd <- sd(beta_boot2[6,])
log_base_treat_sd <- sd(beta_boot2[7,])
log_age_log_base_treat_sd <- sd(beta_boot2[8,])
```

Intercept standard deviation: 3.3647071

Log(age) standard deviation: 1.0080298

Log_base standard deviation: 1.8654386

Treatment standard deviation: 7.3001513

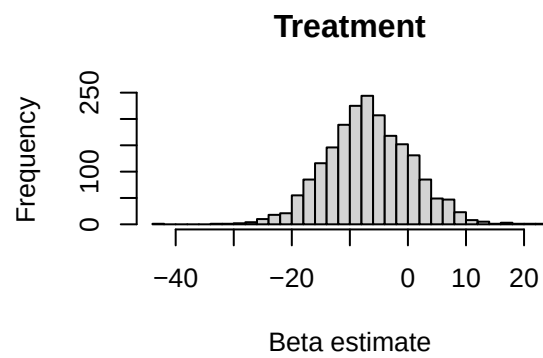
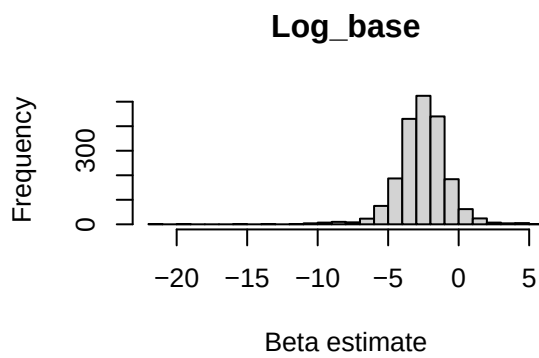
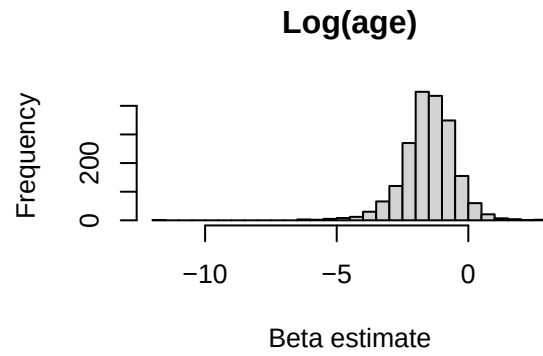
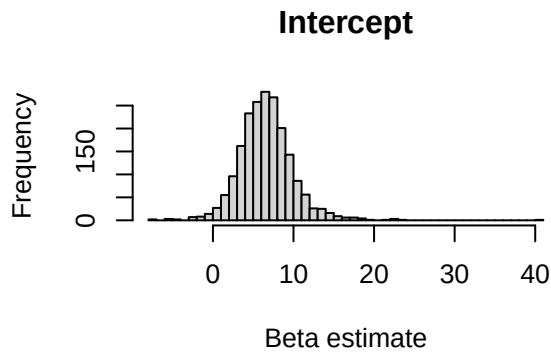
Log(age) * Log_base standard deviation: 0.5571036

Log(age) * Treatment standard deviation: 2.1531741

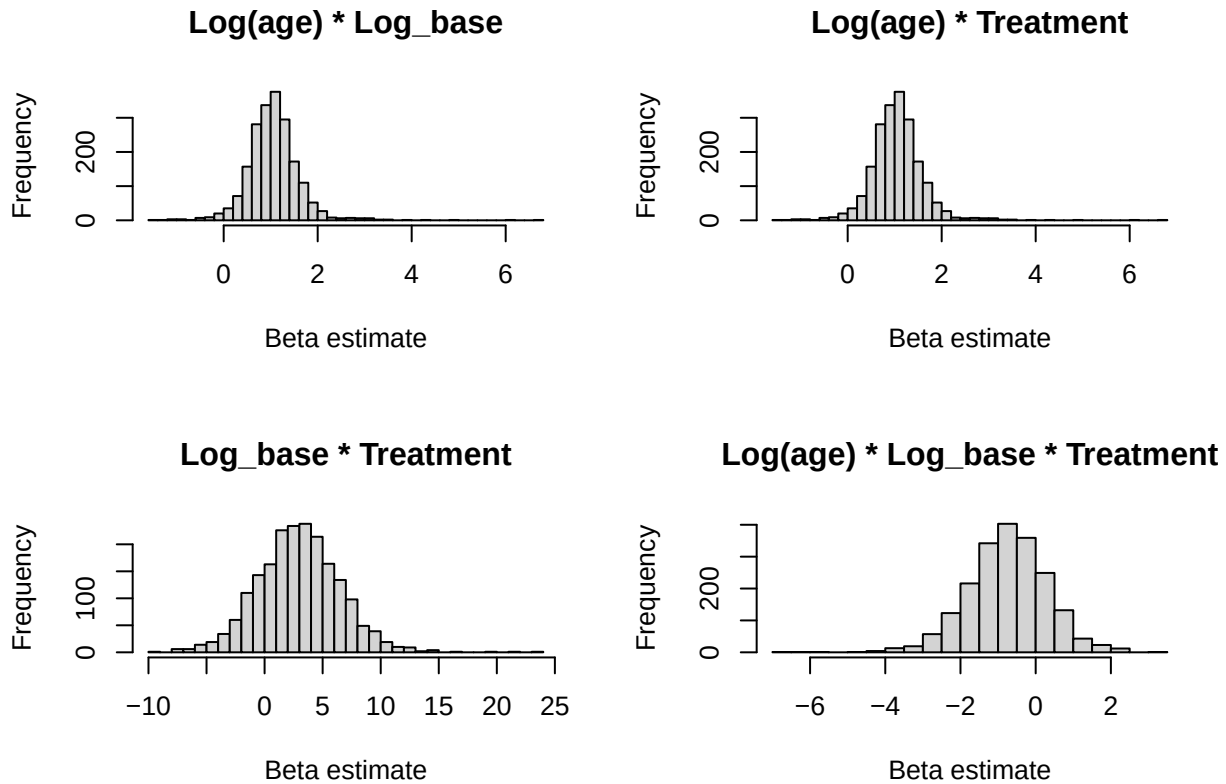
Log_base * Treatment standard deviation: 3.5273709

Log(age) * Log_base * Treatment standard deviation: 1.0543819

```
par(mfrow=c(2,2)) # Histograms of individual terms
hist(beta_boot2[1,], breaks = 35, xlab = "Beta estimate", main = "Intercept")
hist(beta_boot2[2,], breaks = 35, xlab = "Beta estimate", main = "Log(age)")
hist(beta_boot2[3,], breaks = 35, xlab = "Beta estimate", main = "Log_base")
hist(beta_boot2[4,], breaks = 35, xlab = "Beta estimate", main = "Treatment")
```



```
par(mfrow=c(2,2)) # Histograms of interaction terms
hist(beta_boot2[5,], breaks = 35, xlab = "Beta estimate", main = "Log(age) * Log_base")
hist(beta_boot2[5,], breaks = 35, xlab = "Beta estimate", main = "Log(age) * Treatment")
hist(beta_boot2[7,], breaks = 35, xlab = "Beta estimate", main = "Log_base * Treatment")
hist(beta_boot2[8,], breaks = 35, xlab = "Beta estimate", main = "Log(age) * Log_base * Treatment")
```



Generally the bootstrap sampling distributions of the coefficients are similar to the first model. However, `log_base` now appears to be normal and symmetric in support of the normality assumption. This could be due to the Percentile Method's property of working better with skewed data.

```
ci_norm_approx <- rbind( # calculate Normal Approximation CI
  LCI = coefficients(fq2) - qt(0.975, nrow(seiz_total) - 1) * row_sd,
  UCI = coefficients(fq2) + qt(0.975, nrow(seiz_total) - 1) * row_sd
)
round(ci_norm_approx, 2)
```

```
##      (Intercept) log(age) log_base  treat log(age):log_base log(age):treat
## LCI      -0.48    -3.40    -6.17 -22.90          -0.11          -2.26
## UCI      13.00     0.63     1.30  6.32           2.12           6.36
##      log_base:treat log(age):log_base:treat
## LCI          -3.53              -2.98
## UCI          10.59              1.24
```

```
intercept_pci = quantile(beta_boot2[1,], probs = c(0.025, 0.975)) # calculating
log_age_pci = quantile(beta_boot2[2,], probs = c(0.025, 0.975)) # CIs w/ percentile
log_base_pci = quantile(beta_boot2[3,], probs = c(0.025, 0.975)) # method
treat_pci = quantile(beta_boot2[4,], probs = c(0.025, 0.975))
log_age_log_base_pci = quantile(beta_boot2[5,], probs = c(0.025, 0.975))
log_age_treat_pci = quantile(beta_boot2[6,], probs = c(0.025, 0.975))
log_base_treat_pci = quantile(beta_boot2[7,], probs = c(0.025, 0.975))
log_age_log_base_treat_pci = quantile(beta_boot2[8,], probs = c(0.025, 0.975))
```

```
sd_fullm_percentile_ci <- cbind(intercept_pci, # display variable CIs from percentile method together
  log_age_pci,
  log_base_pci,
  treat_pci,
  log_age_log_base_pci,
  log_age_treat_pci,
  log_base_treat_pci,
  log_age_log_base_treat_pci)
round(sd_fullm_percentile_ci, 2)
```

```
##      intercept_pci log_age_pci log_base_pci treat_pci log_age_log_base_pci
## 2.5%          0.58      -3.65      -6.23    -20.66              0.07
## 97.5%         13.83       0.37       0.75      7.62              2.12
##      log_age_treat_pci log_base_treat_pci log_age_log_base_treat_pci
## 2.5%          -2.58          -3.81              -2.87
## 97.5%          5.80           9.86              1.28
```

The confidence intervals differ a little more than for the first model.

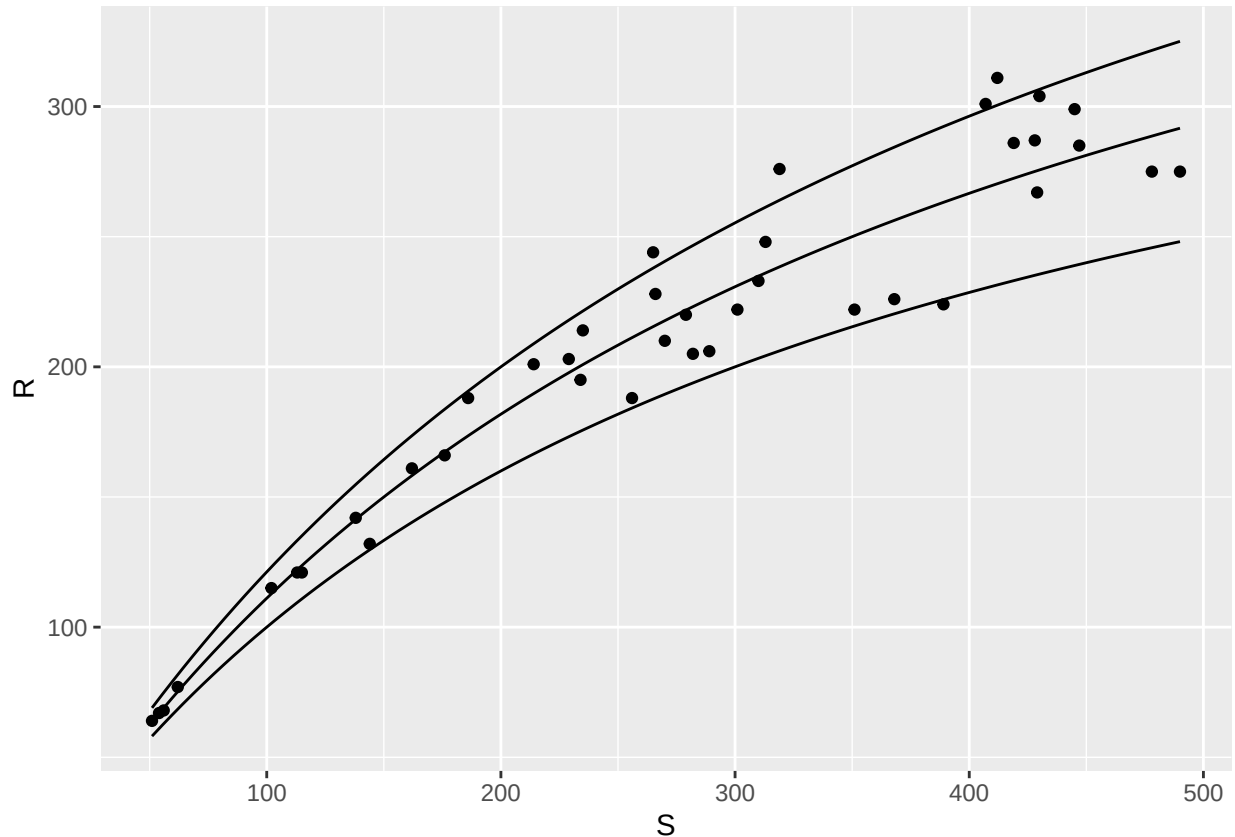
Exercise B: Fish Dataset

1. (4 points) Make a scatterplot of the data and overlay the Beverton-Holt curve for a few different choices of β_1 and β_2 . Hint: Write a function to compute R in the Beverton-Holt model and try values near ($2e-3$, $7e-1$):

```
bh <- function(S, beta1, beta2) {
# TODO: Compute R as a function of S
  R <- 1 / (beta1 + beta2 / S)
  return(R)
}

qplot(S, R, data = fish) +
  stat_function(fun = bh, args = list(beta1 = 2.5e-3, beta2 = 7.5e-1)) +
  stat_function(fun = bh, args = list(beta1 = 2e-3, beta2 = 7e-1)) +
  stat_function(fun = bh, args = list(beta1 = 1.75e-3, beta2 = 6.5e-1))
```

```
## Warning: 'qplot()' was deprecated in ggplot2 3.4.0.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



2. The Beverton-Holt model can be linearized by transforming $R \rightarrow (1/R)$ and $S \rightarrow (1/S)$:
 $(1/R) = \beta_1 + \beta_2(1/S)$

This is a linear model with response variable $(1/R)$ and covariate $(1/S)$. Use least squares regression to fit this model to the fish dataset:

```
bh_lm <- lm(I(1/R) ~ I(1/S), data = fish)
coef(bh_lm)
```

```
## (Intercept)      I(1/S)
## 0.002013231 0.697818835
```

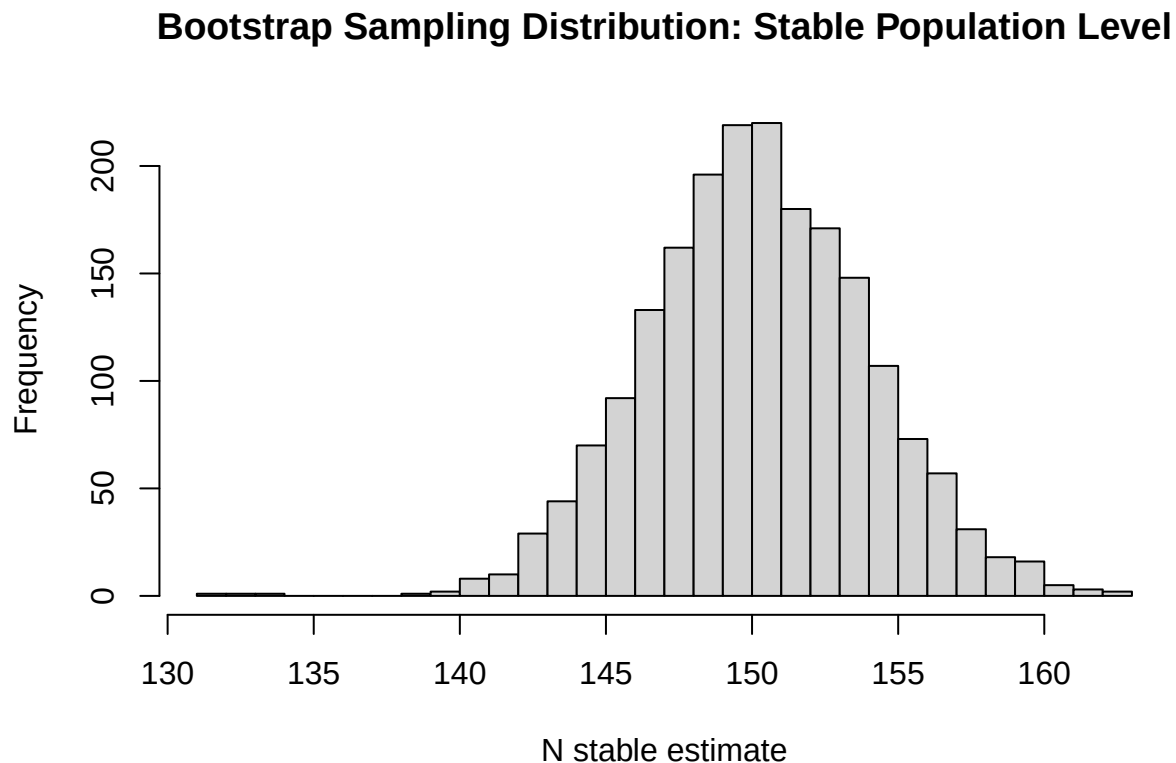
3. Find an estimate for the stable population level, where $R = S$ in the Beverton-Holt model by plugging in your estimated coefficients into the formula for N_{stable} . The resulting estimate is a function of $\hat{\beta}_1$ and $\hat{\beta}_2$.

```
set.seed(1)
n_hat <- function(df) { # Function: get N stable from data frame input
  df = copy(df)
  bh_fit <- lm(I(1 / R) ~ I(1 / S), data = df) # run model for coeffs
  beta2 = coef(bh_fit)[[2]] # extract coeffs
  beta1 = coef(bh_fit)[[1]]
  n_stable = (1 - beta2) / beta1 # calculate and return n stable
  n_stable
}
n_stable <- round(n_hat(fish), 3)
```

An estimate for the N_{stable} is 150.098 $\Rightarrow$ 150.

4. Use the bootstrap to obtain the sampling distribution and standard error for the stable population level. You will need to write a function that encapsulates everything you did in the previous two questions. It should take a dataframe (similar to fish) as input and return an estimated stable population level based on the input:

```
set.seed(1) # set seed for reproducibility
boot_stable <- replicate(2000, {
  boot_sample <- sample_n(fish, size = nrow(fish), replace = TRUE) # sample w/ replacement
  boot_n_stable <- n_hat(boot_sample) # pull and return N stable
  boot_n_stable
})
hist(boot_stable, breaks = 35, xlab = "N stable estimate", main = "Bootstrap Sampling Distribution: Stable Population Level")
```



```
boot_n_stable_mean <- mean(boot_stable) # Calculate mean
round(boot_n_stable_mean, 2)
```

```
## [1] 150.19
```

```
boot_n_stable_se <- sd(boot_stable) # Calculate SE
round(boot_n_stable_se, 2)
```

```
## [1] 3.8
```

5. Read the description of percentile bootstrap confidence intervals from the bootstrap notes and then use the bootstrap to construct a 95% confidence interval for the stable population level.

```
boot_n_stable_percent_ci <- quantile(boot_stable, probs = c(0.025, 0.975))
round(boot_n_stable_percent_ci, 2)
```

```
##      2.5%   97.5%
## 142.90 157.64
```

Using the percentile bootstrap method 95% confidence interval for the stable population level is [142.80, 157.83]. When using the Percentile Method, there are certain advantages including it's simple to compute, it's range preserving (can't be outside the range of the bootstrap distribution), and transformed invariant where we can compute confidence intervals for $\frac{1}{\theta}$ if we want. To show we can do this, we will first exponentiate the n stable CI we found with the Percentile Method.

```
boot_n_stable_percent_ci <- quantile(boot_stable, probs = c(0.025, 0.975))
exp(boot_n_stable_percent_ci)
```

```
##           2.5%           97.5%
## 1.149617e+62 2.889656e+68
```

Then we can run a bootstrap with an exponentiated coefficient $e^{N_{stable}}$

```
set.seed(1) # set seed for reproducibility
exp_boot_stable <- replicate(2000, {
  boot_sample <- sample_n(fish, size = nrow(fish), replace = TRUE) # sample w/ replacement
  boot_n_stable <- exp(n_hat(boot_sample)) # pull and return exp N stable
  boot_n_stable
})
```

And compute our confidence interval using the exponentiated coefficients:

```
exp_boot_n_stable_percent_ci <- quantile(exp_boot_stable, probs = c(0.025, 0.975))
round(exp_boot_n_stable_percent_ci, 2)
```

```
##           2.5%           97.5%
## 1.149762e+62 2.889739e+68
```

As we can see, the confidence intervals are the same showing that the percentile method is indeed a transformed invariant.